Name: MUSABE Leon Armand

LAB REPORT: BETTERCAP MAN-IN-THE-MIDDLE (MITM) ATTACK & ANALYSIS

Technical laboratory report · isolated VirtualBox host-only environment

Table of Contents

1. Executive Summary 2
2. Lab Environment Setup & Connectivity 2
   2.1 Virtual Machine Topology 2
3. Step-by-Step Execution & Technical Findings 3
   3.1 Network Reconnaissance & Host Discovery 3
   3.2 ARP Cache Poisoning (MITM) 3
   3.3 Packet Sniffing & Traffic Analysis 4
   3.4 DNS Spoofing & Fake Web Server Deployment 4
   3.5 HTTP Proxy & JavaScript Injection 5
   3.6 Automated Execution via Caplets 5
5. Security Analysis & Defensive Countermeasures 6
   5.1 Vulnerability Root Cause 6
   5.2 Defensive Countermeasures Matrix 6
6. Conclusion 6

> **Scope.** This report documents a controlled, isolated lab demonstration of network interception techniques and the corresponding defensive controls. All testing was performed within a VirtualBox host-only subnet.

Compact layout edition

1. Executive Summary

This laboratory report provides a technical examination and practical execution of Man-in-the-Middle (MITM) attack vectors using the **Bettercap** framework. The primary objective of this exercise was to simulate network interception within an isolated virtualized environment, evaluating the mechanics of ARP cache poisoning, packet sniffing, DNS spoofing, and HTTP traffic payload injection.

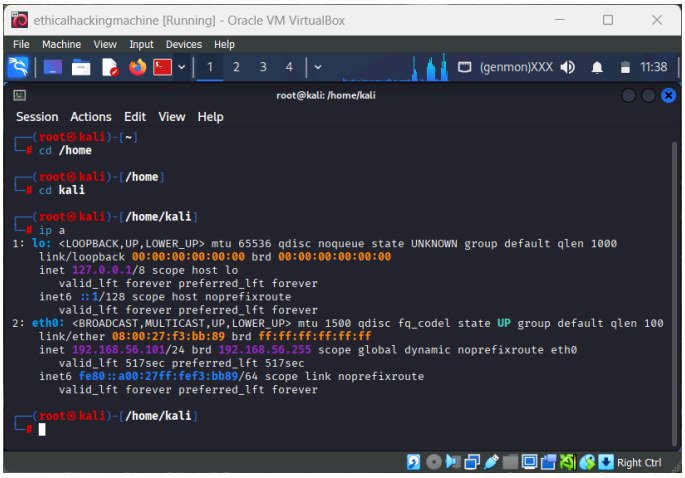
During the exercise, full network interception was established between a Kali Linux attacker virtual machine and a Windows 10 victim virtual machine running on a shared host-only network bridge. Key findings include:

1. **Network Discovery & ARP Spoofing:** Host identification was successfully performed via Bettercap's *net.probe* module. Dual-duplex ARP spoofing was demonstrated, highlighting the vulnerability of unauthenticated ARP resolution.
2. **Traffic Capture & Fallback Analysis:** Active packet capture was achieved and exported to *.pcap* files for Wireshark inspection. Raw socket captures via tcpdump were utilized to ensure complete protocol logging when stream-writing limitations were encountered.
3. **DNS Spoofing & Web Redirection:** Domain name resolution queries were intercepted to redirect target traffic toward a locally hosted custom Python HTTP web server.
4. **Proxying & Payload Injection:** Using custom JavaScript proxy scripts (*inject.js*), real-time HTML response modification was achieved, demonstrating how unencrypted Web traffic can be maliciously altered in transit.
5. **Automation:** The complete attack sequence was consolidated into a reusable script caplet (*.cap*), proving the feasibility of automated MITM deployment.

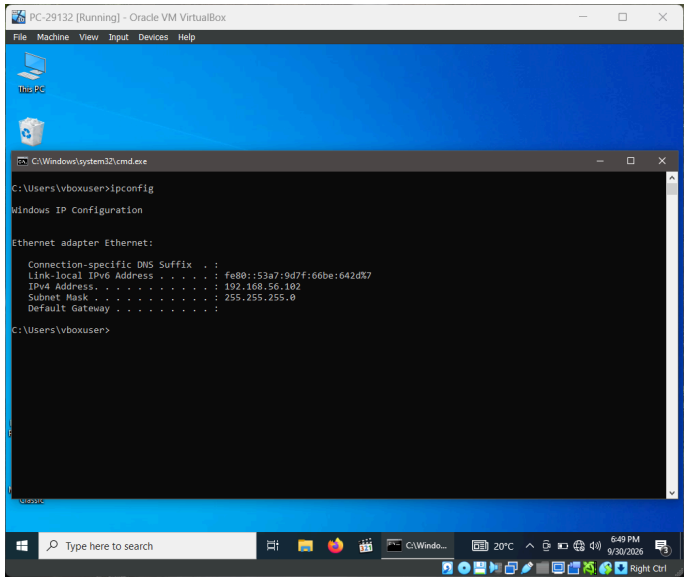
2. Lab Environment Setup & Connectivity

2.1 Virtual Machine Topology

Host Role	Operating System	IP Address	Network Interface
Attacker	Kali Linux	192.168.56.101	Host-only Adapter
Victim	Windows 10 Desktop	192.168.56.102	Host-only Adapter
Gateway (Host)	VirtualBox Host-Only	192.168.56.1	vboxnet0



Kali Linux attacker interface.



Windows victim interface.

Figure 1. Verification of IP addressing and connectivity across the VirtualBox Host-Only subnet (192.168.56.0/24).

3. Step-by-Step Execution & Technical Findings

3.1 Network Reconnaissance & Host Discovery

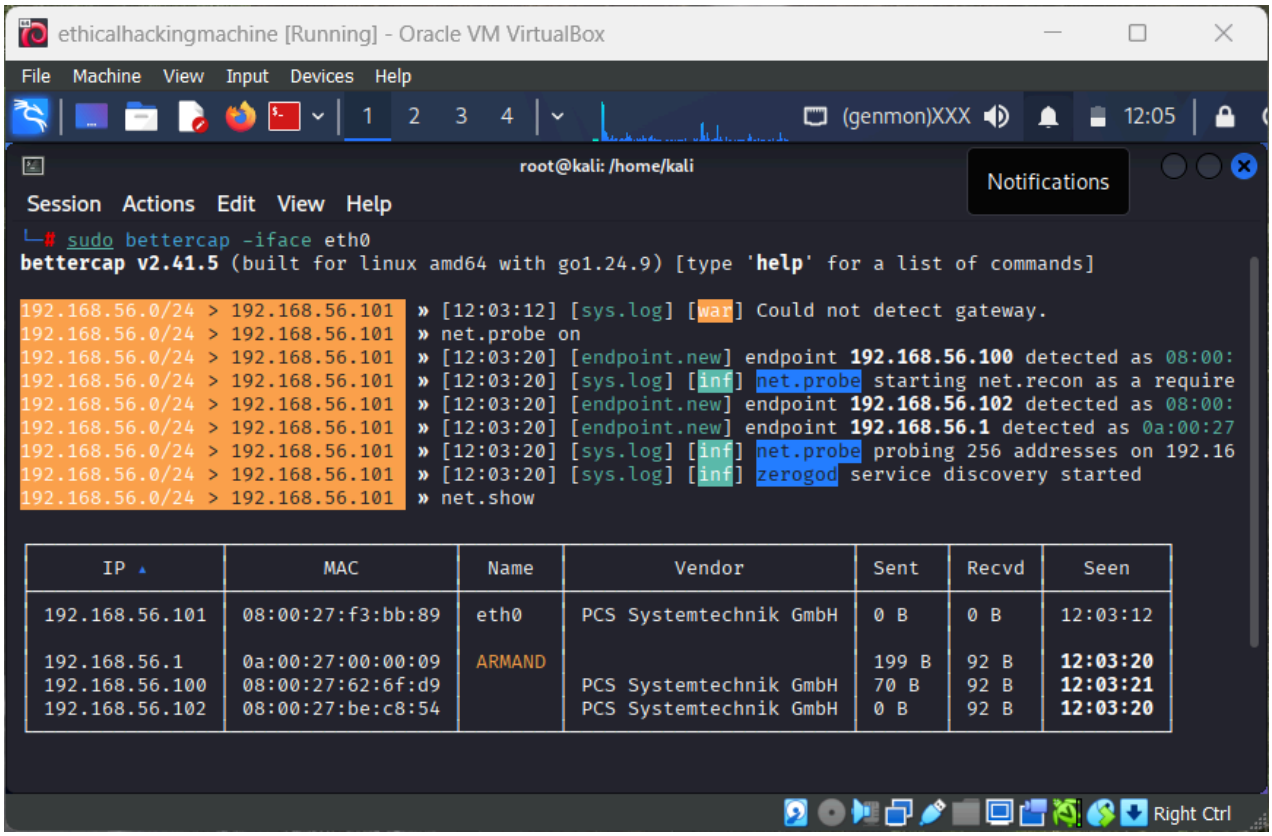


Figure 2. Bettercap reconnaissance showing active detection of the target Windows host (192.168.56.102) and local gateway.

3.2 ARP Cache Poisoning (MITM)

IPv4 forwarding was enabled at the Linux kernel level to route intercepted victim packets seamlessly.

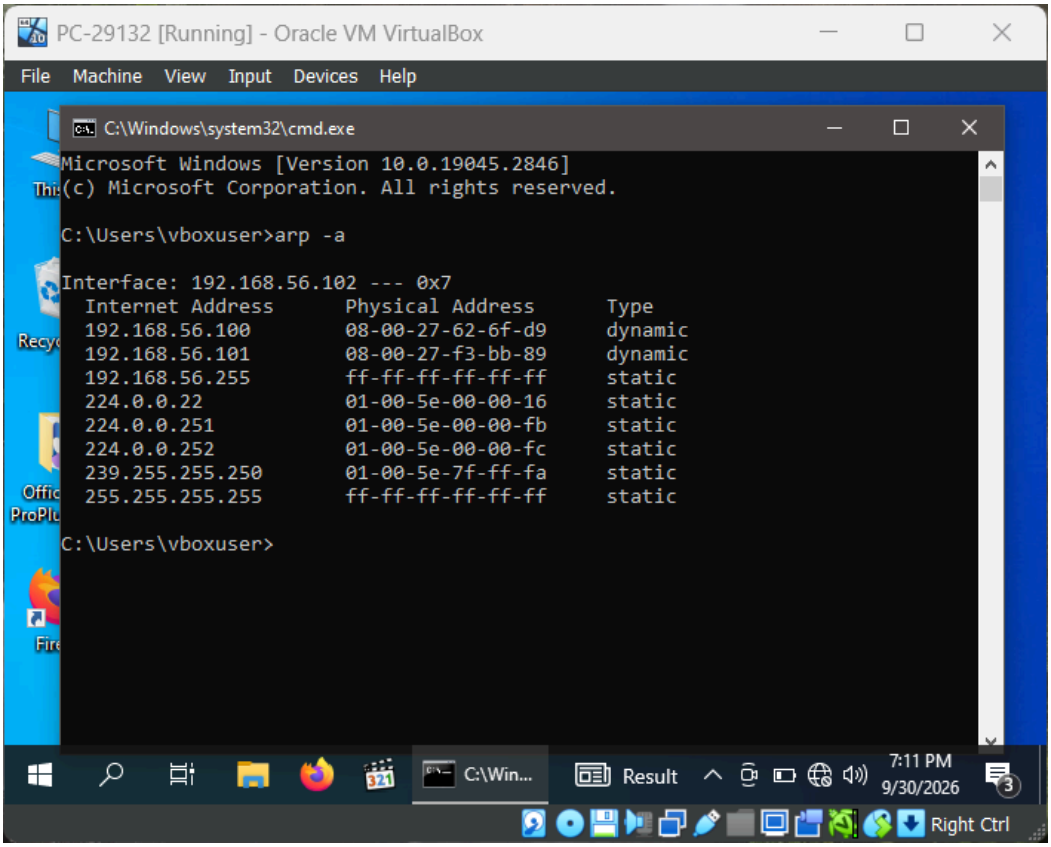


Figure 3. Victim ARP table inspection (*arp -a*) showing the host gateway IP mapped to the Kali Linux MAC address.

3.3 Packet Sniffing & Traffic Analysis

Network traffic was recorded and written to standard *.pcap* formats for protocol inspection in Wireshark.

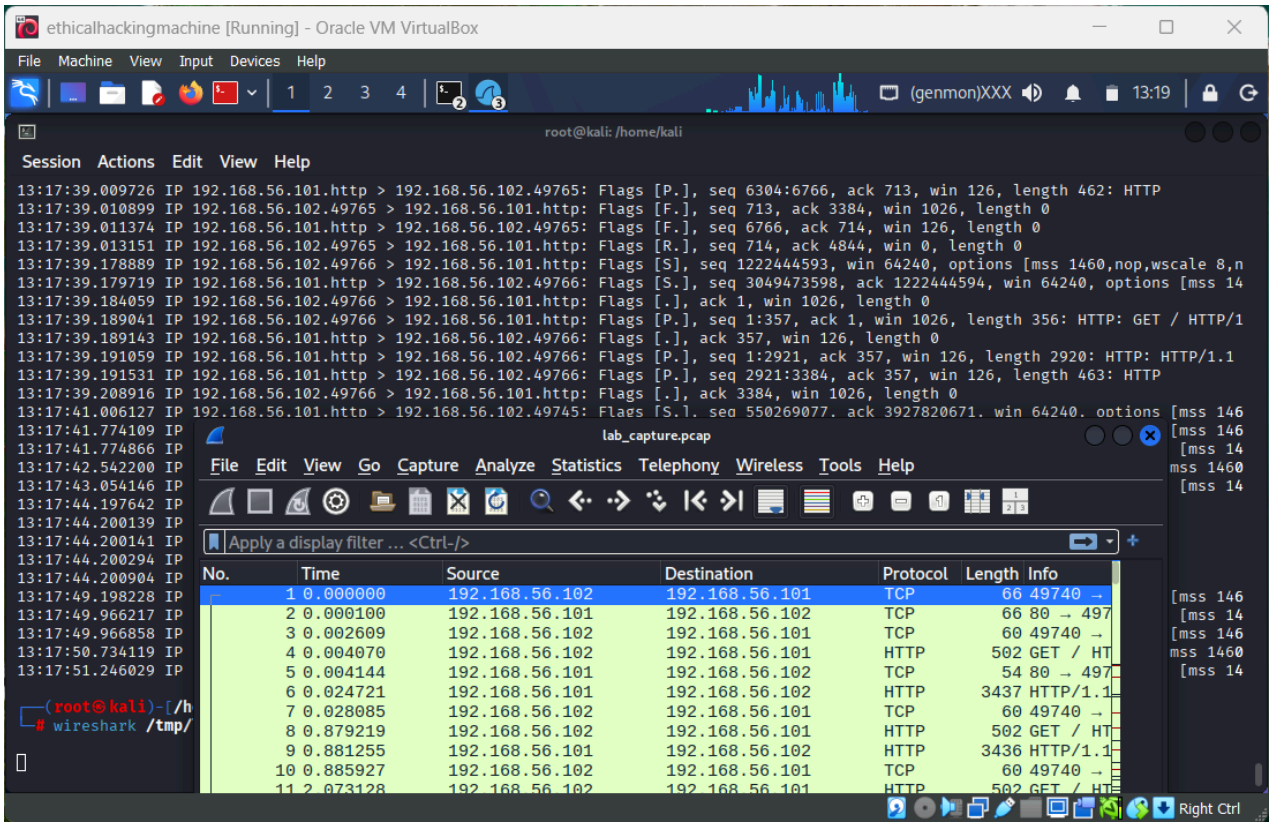
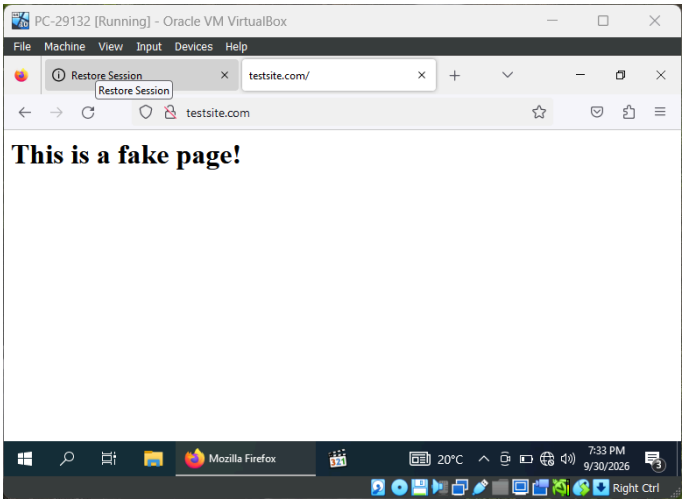


Figure 4. Packet capture and Wireshark inspection of intercepted traffic.

3.4 DNS Spoofing & Fake Web Server Deployment

A local HTTP web server was launched on Kali Linux to host a controlled landing page.



Controlled landing page served to the victim browser.

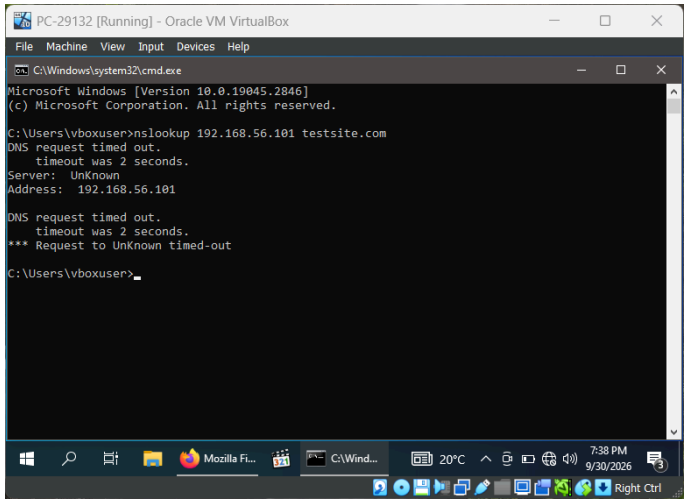


Figure 5. Victim DNS query resolution (*nslookup*) resolving *testsite.com* to 192.168.56.101.

3.5 HTTP Proxy & JavaScript Injection

To demonstrate payload delivery into unencrypted web streams, a custom JavaScript injection script (*/tmp/inject.js*) was crafted.

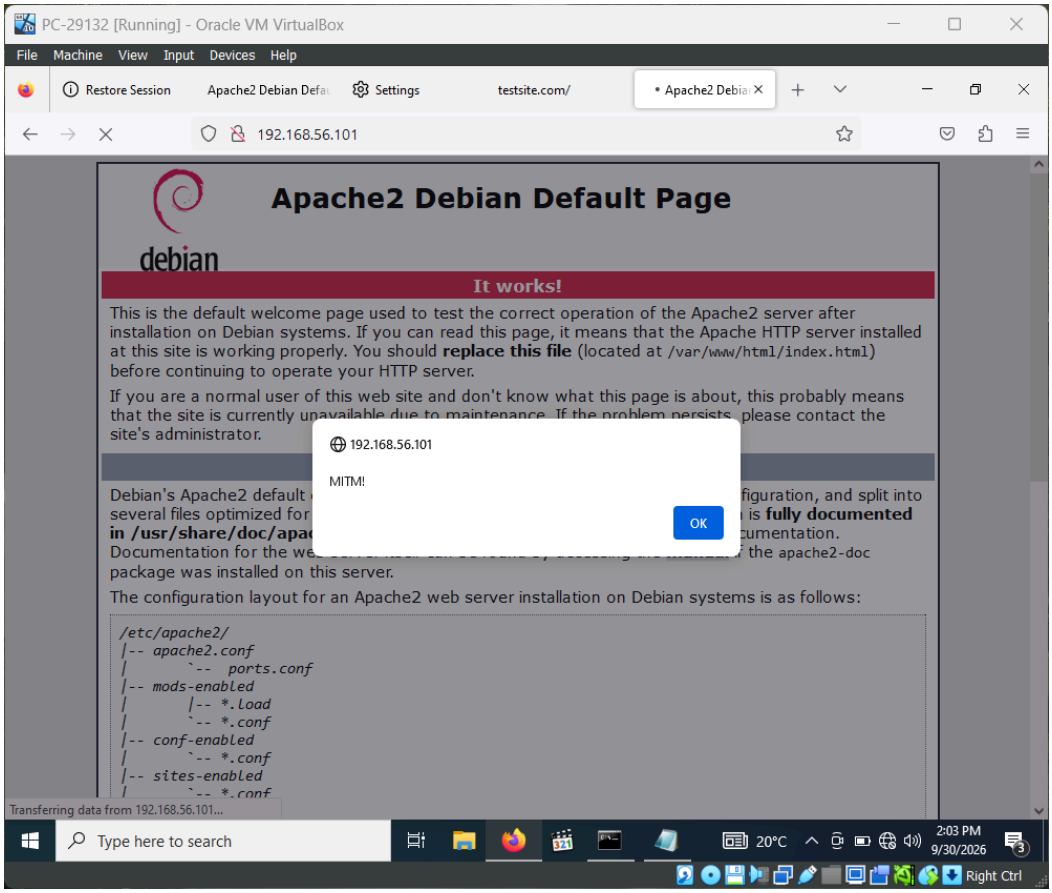


Figure 6. Injected DOM manipulation executed in the victim browser showing the popup notification created by *inject.js*.

3.6 Automated Execution via Caplets

To streamline attack deployment, all configuration commands were consolidated into a caplet automation script (*/home/kali/lab_mitm.cap*).

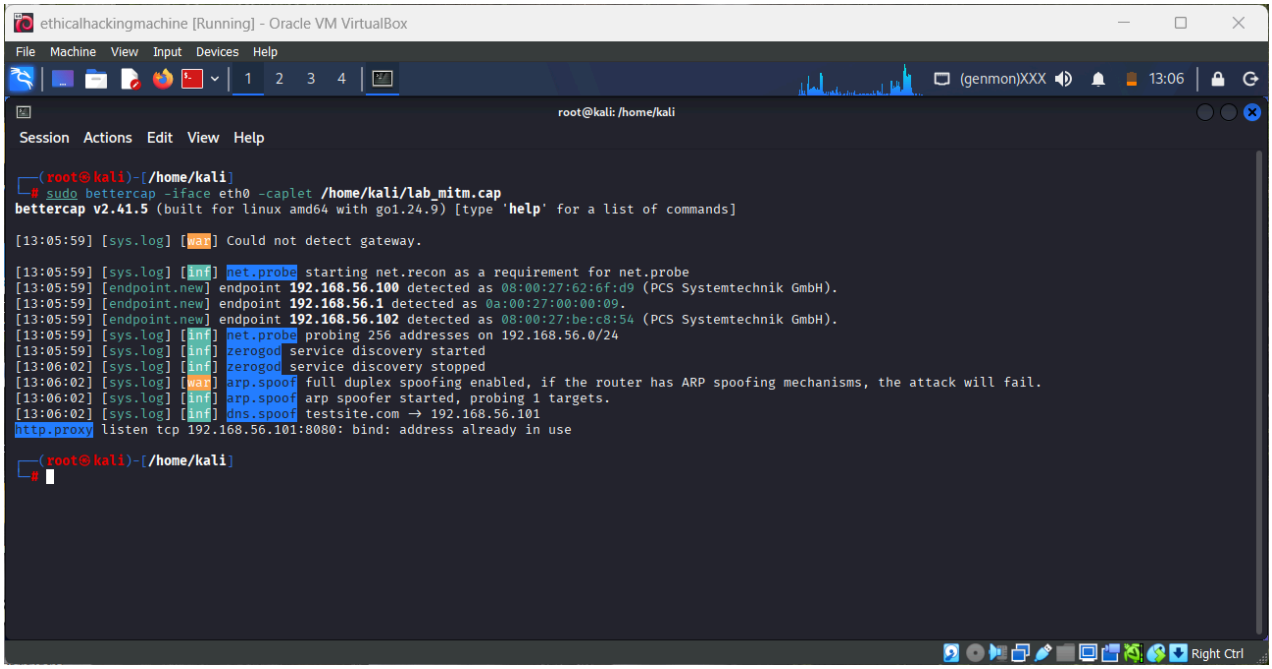


Figure 7. Automated initialization of ARP spoofing, DNS redirection, and HTTP proxying using Bettercap caplet execution.

5. Security Analysis & Defensive Countermeasures

5.1 Vulnerability Root Cause

The success of these attack vectors relies on structural flaws inherent in legacy network protocols:

- **Unauthenticated ARP Resolution:** Ethernet hosts trust incoming gratuitous ARP responses without validation, enabling cache poisoning.
- **Plaintext HTTP Traffic:** Unencrypted HTTP communication permits in-transit packet inspection and dynamic HTML payload alteration.
- **Unsecured Local DNS Resolution:** Unsigned DNS queries can be answered by rogue actors on the local broadcast domain.

5.2 Defensive Countermeasures Matrix

Control	Protective effect
Dynamic ARP Inspection (DAI)	Managed switches validate ARP packets against a trusted DHCP Snooping binding database to drop spoofed replies.
HTTP Strict Transport Security (HSTS)	Browsers enforce encrypted HTTPS connections exclusively, preventing SSL stripping and proxy modification.
Domain Name System Security Extensions (DNSSEC)	Digitally signs DNS records to ensure authenticity and prevent resolution tampering.

6. Conclusion

This laboratory exercise demonstrated the end-to-end execution of Man-in-the-Middle (MITM) attacks using Bettercap within a virtualized network testing environment. Through ARP spoofing, DNS manipulation, and HTTP proxy injection, traffic streams were successfully intercepted and modified. Implementing defensive controls such as Dynamic ARP Inspection (DAI), mandatory HTTPS with HSTS, and encrypted DNS protocols is essential for mitigating these fundamental layer-2 and layer-7 vulnerabilities.